

Cortex-M4 NVIC and SCB Registers

Ankit Srivastava

EE2028

ARMv7-M Architecture Ref Manual

RM0432 STM32L4+ Technical reference manual

PM0214 STM32 Cortex-M4 Programming Manual

Acknowledgement: Slides adapted from Dr Rajesh Panicker



OVERVIEW

- Interrupt Configuration and Status via NVIC and SCB
 - NVIC Registers
 - SCB Registers

INTERRUPT CONFIGURATION/STATUS VIA NVIC AND SCB

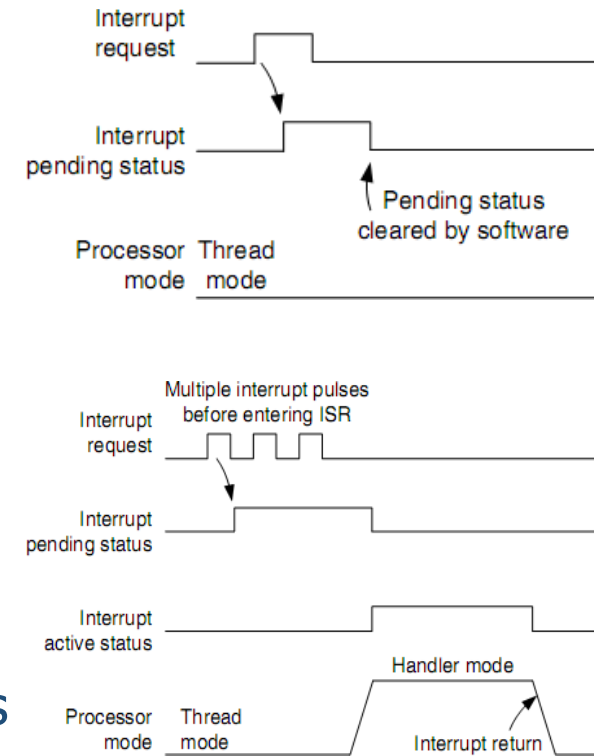
- NVIC registers
 - NVIC_ISER: Interrupt Set Enable Registers
 - NVIC_ICER: Interrupt Clear Enable Registers
 - NVIC_ISPR: Interrupt Set Pending Registers
 - NVIC_ICPR: Interrupt Clear Pending Registers
 - NVIC_IPR: Interrupt Priority Registers
 - NVIC_IABR: Interrupt Active Bit Registers
 - STIR: Software Trigger Interrupt Register
- SCB registers
 - SHPR: System Handler Priority Registers
 - ICSR: Interrupt Control and State Register
 - SHCSR: System Handler Control and State Register
 - VTOR: Vector Table Offset Register
 - PRIGROUP in AIRCR: Priority group in Application Interrupt & Reset Control Register
- Exception-masking registers
 - PRIMASK
 - FAULTMASK
 - BASEPRI

INTERRUPT SET ENABLE REGISTERS (ISER) AND INTERRUPT CLEAR ENABLE REGISTERS (ICER)

- NVIC_ISER (32 bits, RW) is used to
 - Enable interrupts
 - Determine which interrupts are currently enabled and disabled
- NVIC_ICER (32 bits) is used to
 - Disable interrupts
 - Determine which interrupts are currently enabled and disabled
- NVIC can have more than one NVIC_ISER/NVIC_ICER. Each bit of the registers is corresponding to one interrupt
- Enable/Disable an interrupt:
 - To enable an interrupt (set enable bit), write 1 to NVIC_ISER
 - To disable an interrupt (clear enable bit), write 1 to NVIC_ICER

INTERRUPT SET PENDING REGISTERS (ISPR) AND INTERRUPT CLEAR PENDING REGISTERS (ICPR)

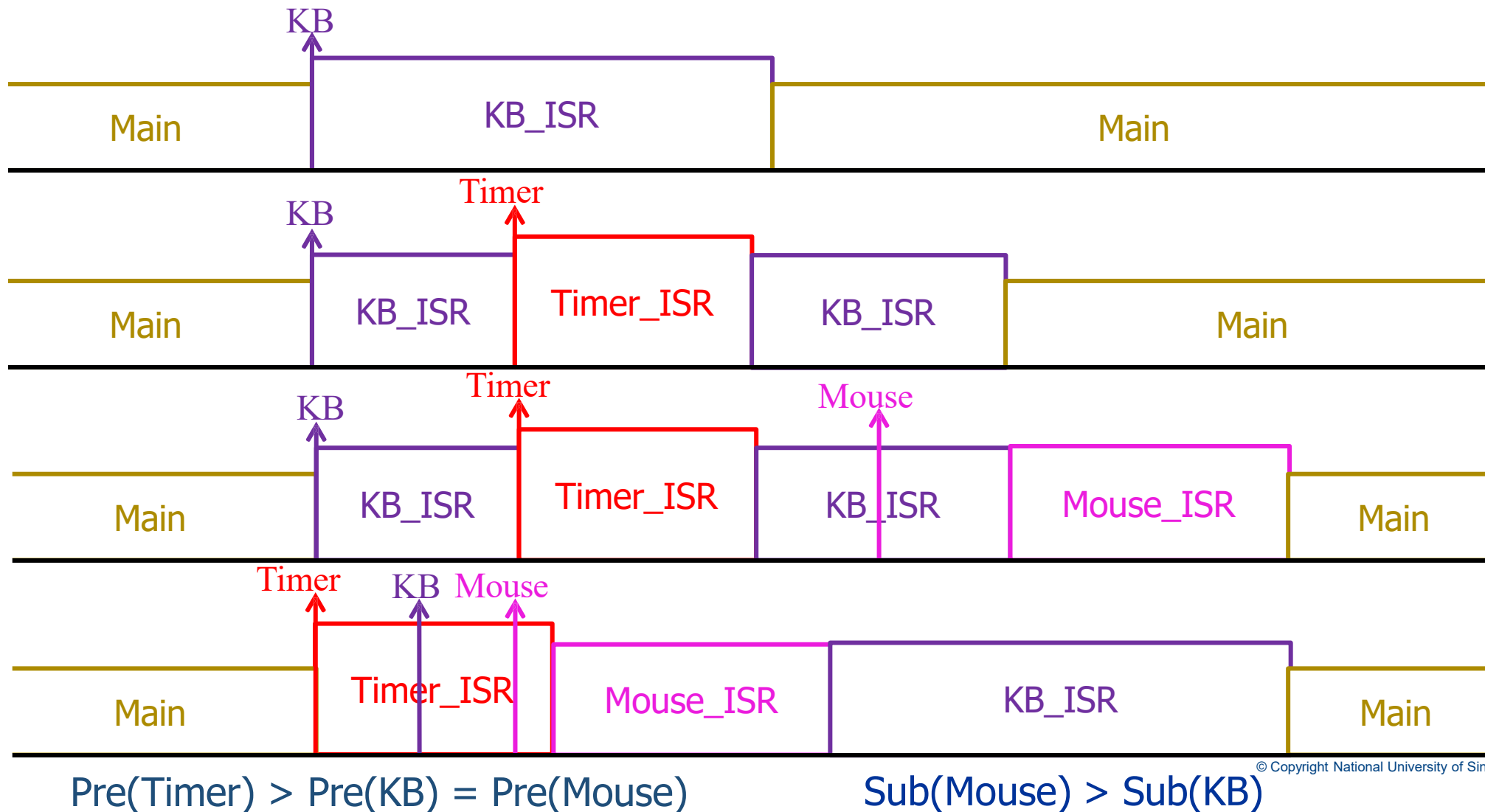
- NVIC_ISPR (32 bits) register is used to
 - Force interrupts into pending state
 - Determine which interrupts are currently pending (R/W)
- NVIC_ICPR (32 bits) register is used to
 - Clear pending interrupts
 - Determine which interrupts are currently pending (R/W)
- NVIC can have more than one NVIC_ISPR/NVIC_ICPR and each bit is corresponding to one interrupt input
- Interrupt pending status can be accessed through NVIC_ISPR/NVIC_ICPR
 - To pend an interrupt, write 1 to corresponding bit in NVIC_ISPR
 - To cancel a current interrupt pending, write 1 to corresponding bit in NVIC_ICPR
 - Writing to NVIC_ISPR has no effect on an interrupt which is already pending
 - Writing to NVIC_ICPR has no effect on the active status of an interrupt.



INTERRUPT PRIORITY REGISTERS (IPR) AND INTERRUPT ACTIVE BIT REGISTERS (IABR)

- NVIC Interrupt Priority Registers (IPRs) are 8-bit registers, though only some of the MSBs are implemented depending on the SoC
- Accessible as `IP[i]`, where $i=0$ to 94 for STM32 as there are 95 external interrupts
- Each register is used to assign a priority level (from 0 to 255) to interrupts: 0 is the highest programmable priority
- Read `NVIC_IABR` to determine which interrupts are active. Each external interrupt has a bit in `NVIC_IABRs`
- When CPU starts an interrupt handler, its active bit is set to 1, and then cleared when return is executed
- If two interrupts are nested, both are active
 - Active = has started, has not finished. An interrupt being active doesn't necessarily mean that the processor is currently executing the ISR
- `NVIC_IABRs` for external interrupts are read only

INTERRUPT PRIORITY REGISTERS (IPR) AND INTERRUPT ACTIVE BIT REGISTERS (IABR)



REGISTERS FOR SYSTEM EXCEPTIONS

- So far, we saw the various NVIC registers which deals with external interrupts. For system exceptions, the corresponding controls are considered a part of the System Control Block (SCB – a core peripheral) rather than NVIC
- SHPR: System Handler Priority Register are used to set Priority of system exceptions with programmable priority levels, accessible as SHP[i], where i=0 to 11 for all CM4 chips

Self-reading

- ICSR: Interrupt Control and State Register (ICSR) can be used to
 - Set and clear the pending status of system exceptions including SysTick, PendSV, and NMI
 - Determine the currently executing exception/interrupt number by reading VECTACTIVE
- SHCSR: System Handler Control and State Register (SHCSR)
 - Usage faults, memory management (MemManage) faults, and bus fault exceptions are enabled by the SHCSR
 - The pending status of faults and active status of most system exceptions are also available from this register

PRIMASK, FAULTMASK AND BASEPRI

- PRIMASK is a 1-bit register (default = 0)
 - When it is set, it allows only NMI, hard fault, and reset exceptions. Other interrupts and exceptions (having PRI ≥ 0) are masked (disabled)
- FAULTMASK is a 1-bit register (default = 0)
 - When it is set, it allows only NMI and reset. All the interrupts and exceptions including hard fault exception (having PRI ≥ -1) are masked
- BASEPRI is an 8-bit unsigned register
 - used to disable all exceptions/interrupts having PRI $\geq$ BASEPRI. Writing a 0 to BASEPRI disables BASEPRI

```
void __enable_irq(); // Clear PRIMASK
void __disable_irq(); // Set PRIMASK
void __set_PRIMASK(uint32_t priMask); // Set PRIMASK to value priMask
uint32_t __get_PRIMASK(void); // Read the PRIMASK value
void __set_FAULTMASK(uint32_t faultMask); // Set or clear FAULTMASK
uint32_t __get_FAULTMASK(void); // Read the FAULTMASK value
void __set_BASEPRI(uint32_t basePri); // Set the base priority register
uint32_t __get_BASEPRI(void); // Return the content of the base priority register
__set_BASEPRI(0x60); // Disable interrupts with priorities from 0x60-0xFF
```

ACCESSING CORTEX-M4 NVIC REGISTERS

CMSIS functions	Description
<code>void NVIC_EnableIRQ(IRQn_Type IRQn)</code>	Enables an interrupt or exception.
<code>void NVIC_DisableIRQ(IRQn_Type IRQn)</code>	Disables an interrupt or exception.
<code>void NVIC_SetPendingIRQ(IRQn_Type IRQn)</code>	Sets the pending status of interrupt or exception to 1.
<code>void NVIC_ClearPendingIRQ(IRQn_Type IRQn)</code>	Clears the pending status of interrupt or exception to 0.
<code>uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn)</code>	Reads the pending status of interrupt or exception. This function returns non-zero value if the pending status is set to 1.
<code>uint32_t NVIC_GetActive(IRQn_Type IRQn);</code>	Read the active bit for an external interrupt
<code>void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)</code>	Sets the priority of an interrupt or exception with configurable priority level to 1.
<code>uint32_t NVIC_GetPriority(IRQn_Type IRQn)</code>	Reads the priority of an interrupt or exception with configurable priority level. This function return the current priority level.
<code>void NVIC_SetPriorityGrouping(uint32_t PriorityGroup);</code>	Set the Priority Grouping in NVIC Interrupt Controller
<code>uint32_t NVIC_EncodePriority (uint32_t PriorityGroup, uint32_t PreemptPriority, uint32_t SubPriority);</code>	Encode the priority for an interrupt. The returned priority value can be used for NVIC_SetPriority(...) function

CONCLUSION

- Interrupt Configuration and Status via NVIC and SCB
 - NVIC Registers
 - ISER and ICER
 - ISPR and ICPR
 - IPR
 - IABR
 - SCB Registers
 - SHPR
 - ICSR
 - SHCSR
 - PRIMASK, FAULTMASK and BASEPRI
- CMSIS functions